

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Numerik 5:

Digitale Simulation

Inhalte

5. Digitale Simulation

- 5.1 Einführung
- 5.2 Klassische Integrationsverfahren
- 5.3 Einschrittverfahren
- 5.4 Mehrschrittverfahren
- 5.5 Übersicht der Verfahren
- 5.6 Spezielle Probleme

5. Digitale Simulation – 5.1 Einführung

- **Was ist die Bedeutung der Simulation technischer Systeme?**

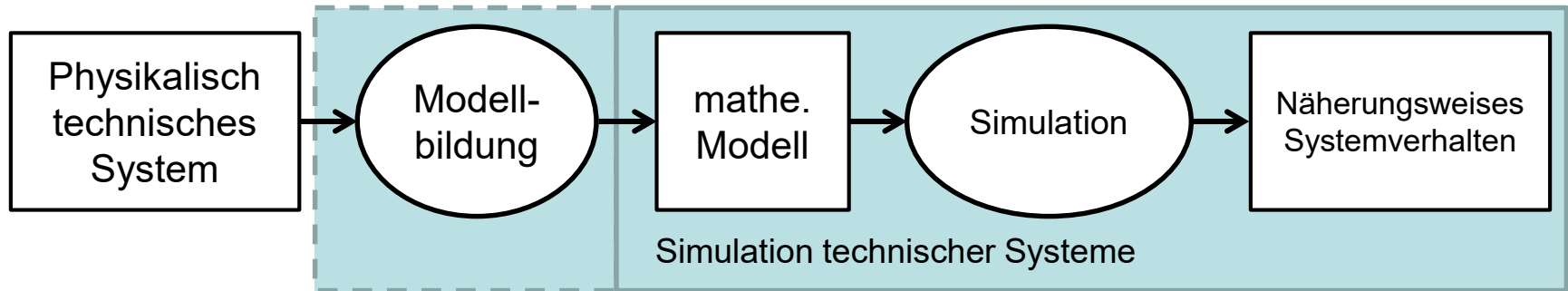
Die Simulation ist heute nicht mehr aus den Ingenieurwissenschaften wegzudenken. So gibt es kein erstzunehmendes technisches Problem oder System, das nicht zunächst mittels Simulation untersucht, bzw. entwickelt wird. Bei komplexen Produkten – wie z. B. Kraftfahrzeugen – kommen disziplinspezifisch unterschiedliche Simulationstechniken (E-Technik, Maschinenbau, usw.) für alle Komponenten (Motor, Elektronik, Aerodynamik, usw.) und über den gesamten Lebenszyklus des Produkts (Forschung, Entwicklung, Produktion, Entsorgung, usw.) zum Einsatz. Für die Simulation bei unterschiedlichen Problemstellungen wurden spezialisierte Ansätze und Lösungen entwickelt (z. B. Schaltungssimulation, FEM, Echtzeitsimulation).

5. Digitale Simulation – 5.1 Einführung

Die verfügbaren Simulationstools sind heute so komfortabel und zuverlässig, dass sie dem Ingenieur ein effektives Arbeiten ermöglichen, ohne dass er sich um die Funktion des Simulationstools kümmern muss. Dadurch wird aber auch oft der falsche Eindruck vermittelt, dass der Benutzer keine besonderen Kenntnisse über Simulationsverfahren benötigt. Tatsächlich ist eine gute Kenntnis der simulationstechnischen Grundbegriffe schon Voraussetzung, um selbst die grundlegendsten Simulationsparameter für die geplante Simulationsaufgabe anpassen zu können. Müssen darüber hinaus z. B. Toolkopplungen vorgenommen, Modelle für die Simulationsaufgabe optimiert oder spezielle Algorithmen angewendet werden, bedingt dies ein tiefgreifendes Wissen über Simulationsverfahren einschließlich der zugrundeliegenden numerischen Mathematik. Die Vorlesung „Rechnergestützte Numerik und Simulationstechnik“ vermittelt sowohl die theoretischen Grundlagen der numerischen Mathematik und digitalen Simulation als auch konkrete Verfahren und den praktischen Umgang mit Simulationstools, wobei die Übungen am Beispiel von Matlab/Simulink durchgeführt werden. Die Studierenden sollen durch die Vorlesung in die Lage versetzt werden, Simulationsaufgaben und allgemeine numerische ingenieurwissenschaftliche Programmieraufgaben effizient umzusetzen, wobei das Gelernte auf andere Simulationsumgebungen und Programmiersprachen übertragbar ist.

5. Digitale Simulation – 5.1 Einführung

Einführung in die digitale Simulation



Wichtig: Die Modellbildung beeinflusst das Modell und damit die Art der Simulation
→ Es ist keine vollkommen isolierte Betrachtung der Simulation möglich.

Unterscheidung:

- *Simulations-Werkzeuge:* Das Modell liegt (zumindest als Blockdiagramm) vor und wird über die implementierten Algorithmen simuliert (Beispiel: Simulink ohne Zusatzbibliotheken wie z. B. SimPowerSystems)
- *Modellierungswerkzeuge:* Erzeugung eines mathematischen Modells aus einer z. B. topologischen oder geometrischen Darstellung heraus (Beispiel Pspice, Simplorer, Maxwell3D, usw.) - i. d. R. gekoppelte mit dem Simulationswerkzeug.

5. Digitale Simulation – 5.1 Einführung

Wichtige Kriterien für Simulationsverfahren:

- (Numerische) Stabilität
 - Genauigkeit
 - Rechenaufwand (Rechenzeit)
 - Algorithmisierbarkeit
-
- Verhalten bei steifen Systemen
 - Verhalten bei nicht-linearen Systemen

5. Digitale Simulation – 5.1 Einführung

Man unterscheidet...

„Off-line“-Simulation (keine Echtzeit)

- Systemanalyse/-synthese
- Entwurfsvalidierung/-optimierung

„On-line“-Simulation (Echtzeit)

- Einbindung des Menschen
- Einbindung technischer Echtteile (HIL oder RCP)

Analoge Simulation:

- Analogrechner (chopper-stabilisiert)
- Parallel Bearbeitung
- Zeit- u. wertekontinuierliche Signale
- Hohe Kosten...
 - beim Einrichten und
 - beim Invest
- nur Simulation
- Schlechte Reproduzierbarkeit und Dokumentierbarkeit
- Echtzeitfähig

- Heute keine Bedeutung mehr

Digitale Simulation

- Digitalrechner
- Sequentielle Bearbeitung
- Zeit- u. wertediskrete Signale
- Niedrige Kosten durch...
 - Preiswerte, performante Software
 - Standardsysteme (z. B. PC)
- Modellbildung u. Simulation
- 100% reproduzierbar, gute Dokumentierbarkeit
- bedingt echtzeitfähig

- Heute Standard

5. Digitale Simulation – 5.1 Einführung

Anwendungsgebiete

„Verteilte“ Prozesse → partielle Differentialgleichungen:

- Stationäre oder instationäre Strömungsprozesse
- Stationäre oder instationäre elektromagnetische Felder
- Stationäre oder instationäre Spannungsfelder

„Konzentrierte“ Prozesse → Differentialgleichungen mit konzentrierten Parametern:

- Kontinuierliche analoge Prozesse:
 - Elektrische Schaltungen
 - Mechanische Mehrkörpersysteme
 - ...
- Diskrete Prozesse:
 - Anlagenplanung/-optimierung

5. Digitale Simulation – 5.1 Einführung

Systembeschreibung (mathematisches Modell)

Die allgemeine implizite Form einer (nichtlinearen) Differentialgleichung

$$f_1(y, \dot{y}, \ddot{y}, \dots, y^{(m)}, u) = 0 \quad \text{mit } y \in \mathbb{R}^n$$

lässt sich als explizite gewöhnliche Differentialgleichung (ODE – Ordinary Differential Equation) schreiben, wenn sie sich nach der höchsten Potenz von x auflösen lässt:

$$y^{(m)} = f_2(y, \dot{y}, \ddot{y}, \dots, y^{(m-1)}, u) \quad (1.1)$$

In diesem Fall lässt sich auch die Zustandsdifferentialgleichung aufstellen:

$$\dot{x} = \begin{bmatrix} \begin{bmatrix} 0 & I & & 0 \\ \vdots & & \ddots & \\ 0 & 0 & & I \end{bmatrix} x \\ f_2(x) \end{bmatrix} \quad \text{mit} \quad x = \begin{bmatrix} y \\ \dot{y} \\ \vdots \\ y^{(m-1)} \end{bmatrix} \in \mathbb{R}^{n \cdot m} \quad (1.2)$$

Hinweis: Kann man nicht so verfahren, dann liegt zunächst eine sogenannte Differential-algebraische Gleichung (DAE – Differential Algebraic Equation) vor, s. Kap. 1.5.

5. Digitale Simulation – 5.1 Einführung

Ein System das einer gewöhnlichen Differentialgleichung gehorcht, lässt sich also in Zustandsdarstellung schreiben:

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{f}(\mathbf{x}, \mathbf{u}) \\ \mathbf{y} &= \mathbf{g}(\mathbf{x}, \mathbf{u})\end{aligned}\quad (1.3)$$

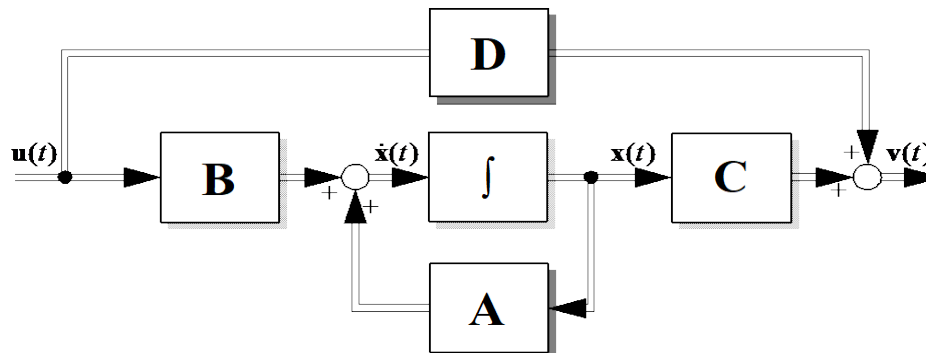
Hinweis: Alle kontinuierlichen Simulink-Blöcke sind in dieser Form implementiert und auch benutzerdefinierte kontinuierliche Systeme lassen sich so darstellen (s. Simulink s-Functions)

Im Fall eines linearen Systems lässt sich (1.3) schreiben als:

$$\begin{aligned}\dot{\mathbf{x}}(t) &= \mathbf{A}(t)\mathbf{x}(t) + \mathbf{B}(t)\mathbf{u}(t) \\ \mathbf{y}(t) &= \mathbf{C}(t)\mathbf{x}(t) + \mathbf{D}(t)\mathbf{u}(t)\end{aligned}\quad (1.4)$$

Bei zeitinvarianten linearen Systemen (LTI) entfällt die Zeitabhängigkeit von $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, $\mathbf{D}$.

Wirkungsplandarstellung:



5. Digitale Simulation – 5.1 Einführung

Ziel der digitalen Simulation:

Ermittlung von zeitdiskreten Werten für x und y zu den Zeitpunkten $t_1, t_2, t_3 \dots t_n$ bei einem gegebenen Anfangswertproblem (System) mit

$$\begin{aligned} \dot{x}(t) &= \mathbf{f}(x(t), \mathbf{u}(t), t) \\ y(t) &= \mathbf{g}(x(t), \mathbf{u}(t), t) \end{aligned} \quad \text{mit} \quad x(0) = x_0 ,$$

also einen Mittels Digitalrechner darstellbaren (algorithmisierbaren) Formalismus der

$$x(t_k) = f_D(\dots) \quad \text{bzw.} \quad y(t_k) = g_D(\dots)$$

liefert. Dies führt praktisch immer auf eine Differenzengleichung der Form

$$x(t_{k+1}) = f_D(x(t_k), x(t_{k-1}), \dots, x(t_{k-n}), u(t_k), \dots, u(t_{k-m}))$$

bzw. eine äquivalent darstellbares Differenzengleichungssystem.

5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Klassische Integrationsverfahren:

In der Praxis wird für die Implementierung von Simulations-Tools in der Regel auf klassische Integrationsverfahren zurückgegriffen, da diese gegenüber anderen Methoden (*Filtermethode, augmentierte Systeme*) eine gute Algorithmisierbarkeit und Verallgemeinerbarkeit aufweisen .

Idee:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t), t) \quad \text{mit} \quad \mathbf{x}(t_k) = \mathbf{x}_k$$

Diskretisierung:

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + \int_{t_k}^{t_{k+1}} \mathbf{f}(\tau) d\tau \approx \mathbf{x}(t_k) + \text{?}$$

→ Approximation des Integrals - also numerische Integration

Die einzelnen Verfahren unterscheiden sich durch die Art der numerischen Auswertung des Integrals.

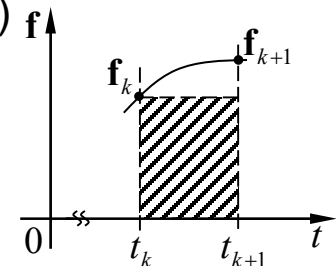
5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Einige klassische numerische Integrationsverfahren:

Euler-Verfahren (Vorwärts-Euler, expliziter Euler, klassisches Eulerv.)

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + T \mathbf{f}(\mathbf{x}(t_k), \mathbf{u}(t_k), t_k) \quad \text{mit} \quad T = t_{k+1} - t_k$$

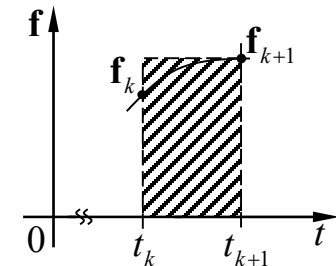
geschrieben $\mathbf{x}_{k+1} = \mathbf{x}_k + T \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, t_k)$



Rückwärts-Euler (impliziter Euler, Backward Differential Equation - BDF, First Difference)

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + T \mathbf{f}(\mathbf{x}(t_{k+1}), \mathbf{u}(t_{k+1}), t_{k+1}) \quad \text{mit} \quad T = t_{k+1} - t_k$$

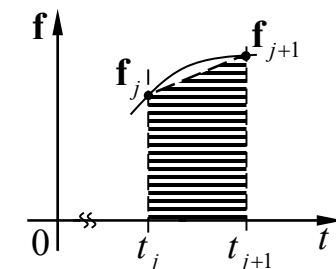
geschrieben $\mathbf{x}_{k+1} = \mathbf{x}_k + T \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1})$



Trapezregel (Tustin)

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + \frac{T}{2} (\mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1}) + \mathbf{f}(\mathbf{x}(t_k), \mathbf{u}(t_k), t_k))$$

geschrieben $\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{T}{2} (\mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1}) + \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, t_k))$



5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Allgemeine Form:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta(\mathbf{x}_{k+1}, \mathbf{x}_k, \mathbf{x}_{k-1}, \dots, \mathbf{x}_{k-n}, \mathbf{u}_{k+1}, \mathbf{u}_k, \dots, \mathbf{u}_{k-n}, t_{k+1}, t_k, \dots, t_{k-n})$$

Wichtige Kriterien:

Implizite / explizit Verfahren:

Implizite Verfahren: $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta(\mathbf{x}_{k+1}, \mathbf{x}_k, \dots)$ die rechte Seite enthält $\mathbf{x}_{k+1}$.

Explizite Verfahren: $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta(\mathbf{x}_k, \dots)$ die rechte Seite enthält $\mathbf{x}_{k+1}$ nicht.

- Explizite Verfahren lassen eine direkte Berechnung in jedem Rechenschritt/Abtastschritt zu.
- Für implizite Verfahren müssen spezielle Methoden angewendet werden (s. u.).

Hinweis: Das Euler-Verfahren (Vorwärts-Euler) ist ein *explizites Verfahren*, während der *Rückwärts-Euler* und die *Trapez-Regel implizit* sind (s. o.).

5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

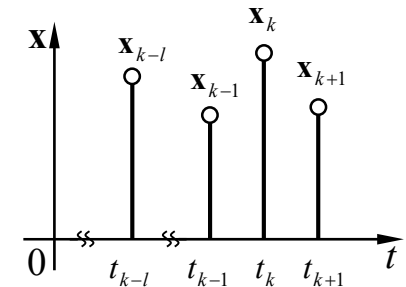
Einschritt- / Mehrschrittverfahren:

Numerische Integrationsverfahren bei denen die Integration über mehrere Abtastschritte (zeitdiskrete Stützstellen) hinweg erfolgen werden als **Mehrschrittverfahren** bezeichnet:

$$x(t_{k+1}) = x(t_{k-n}) + \int_{t_{k-n}}^{t_{k+1}} f(\tau) d\tau$$

Dies bedeutet, das auch die aus dem konkreten Verfahren resultierende Differenzengleichung „ältere“ Vergangenheitswerte von x enthält:

$$x_{k+1} = x_k + k(x_{k+1}, x_k, x_{k-1}, \dots, x_{k-n}, \dots)$$



Numerische Integrationsverfahren bei denen die Integration nur über einen Abtastschritte hinweg erfolgen werden als **Einschrittverfahren** bezeichnet:

$$x(t_{k+1}) = x(t_k) + \int_{t_k}^{t_{k+1}} f(\tau) d\tau$$

Die Differenzengleichung enthält keine „älteren“ Vergangenheitswerte:

$$x_{k+1} = x_k + k(x_{k+1}, x_k, u_{k+1}, u_k)$$

5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Einschritt- vs. Mehrschrittverfahren:

- *Mehrschrittverfahren* haben gegenüber *Einschrittverfahren* numerisch häufig günstiger Eigenschaften (z. B. bei steifen Systemen) bei teilweise geringerem Rechenaufwand.
- *Mehrschrittverfahren* benötigen beim Simulationsstart eine “Anlaufrechnung” über n Schritte bis genügend Vergangenheitswerte zur Verfügung stehen. Die “Anlaufrechnung” erfolgt üblicherweise Mittels eines *Einschrittverfahren*.
- *Mehrschrittverfahren* benötigen die Anlaufrechnung teilweise auch nach strukturellen Änderungen im Modell.

Hinweis 1: Für Simulationszwecke werden häufig die „selbststartenden“ *Einschrittverfahren* ($n = 0$) bevorzugt.

Hinweis 2: Das *Euler-* (Vorwärts-Euler), das *Rückwärts-Euler-Verfahren* und die Trapezregel sind *Einschrittverfahren* (s. o.).

5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Stabilität:

Eine der wichtigsten Eigenschaften eines numerischen Integrationsverfahrens ist seine Stabilität, d. h. dass die Zustandsvariablen (und entsprechend auch die Ausgangswerte) bei begrenzter, konstanter Eingangsgröße gegen einen Endwerte streben (konvergieren).

Für den allgemeinen nicht-linearen Fall sind keine allgemeingültigen Aussagen zur Konvergenz eines Integrationsverfahrens möglich. Daher wird die Stabilität eines Integrationsverfahrens immer am linearen Fall bewertet. Die grundsätzlich Übertragbarkeit der Bewertung auf nichtlineare Systeme wird z. B. durch die mögliche Linearisierung in der Ruhelage deutlich. Häufig wird als Testgleichung

$$\dot{x} = \lambda \cdot x \quad \text{mit} \quad \lambda \in \mathbb{C}$$

verwendet wird → System erster Ordnung ohne Eingangsgröße aber komplexer Eigenwert.

5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Stabilität des Euler-Verfahrens:

Wird $\dot{x} = \lambda \cdot x$ mit $\lambda \in \mathbb{C}$ auch das Eulerverfahren angewendet, erhält man als Rekursionsvorschrift:

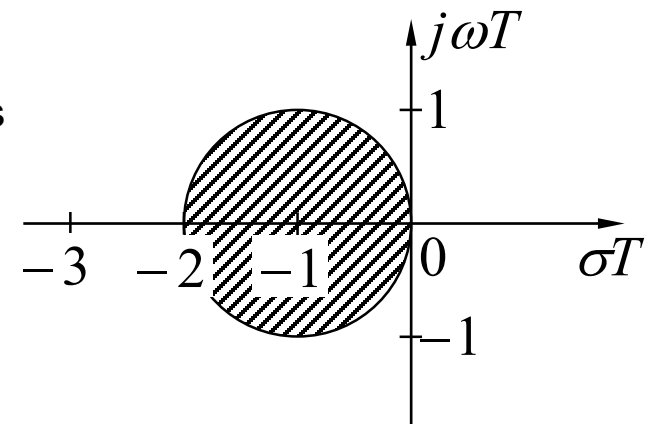
$$x_{k+1} = x_k + \lambda T \cdot x_k = (1 + \lambda T)x_k = ax_k$$

Es wird sofort deutlich, dass das Verfahren nur dann konvergiert (hier gegen 0), wenn der Betrag von $a \in \mathbb{C}$ kleiner 1 ist, also gilt:

$$|a| = |1 + \lambda T| < 1 .$$

Komplexe Eigenwerte λ für die das Eulerverfahren stabil ist, müssen daher in der komplexen Ebene in einem Kreis des Radius $1/T$ um den Wert -1 liegen (s. Zeichnung). Dieser Bereich heißt **Stabilitätsbereich** des Verfahrens.

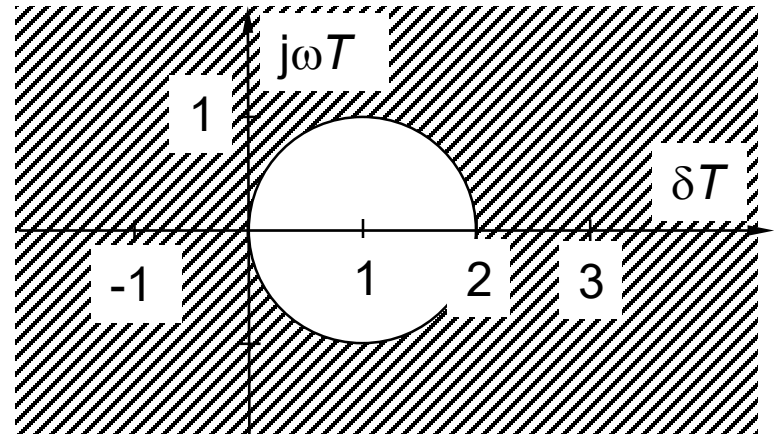
Hinweis: Liegen Eigenwerte außerhalb des Kreises (z. B. zu hohe Eigenfrequenzen) kann der Kreis durch Reduzierung von T (Abtastzeit) vergrößert werden.



5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Stabilität des Rückwärts-Euler-Verfahrens:

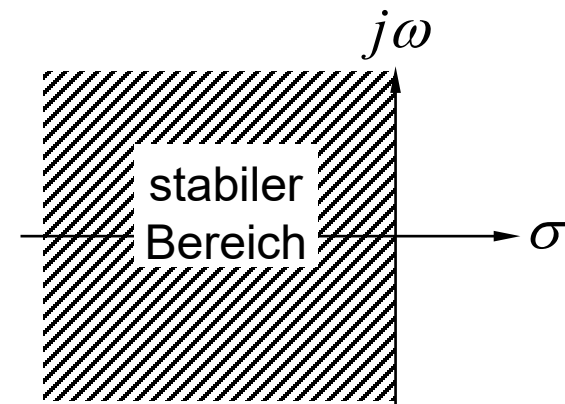
Stabilitätsbereich: Komplette komplexen Ebene außer einem Kreis mit Radius $1/T$ um den Wert $+1$.



Stabilität der Trapezregel:

Stabilitätsbereich: Gesamte linke Hälfte der s-Ebene.

Ist die zu integrierende Differentialgleichung stabil, ist auch die zugehörige Differenzengleichung stabil.



5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

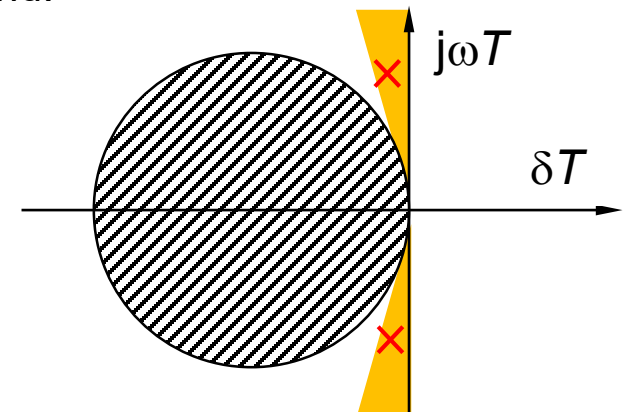
Bewertung der Stabilitätsbereiche:

1. Vorwärts-Euler-Verfahren:

Das Vorwärts-Euler-Verfahren ist das einfachste aller numerischen Integrationsverfahren und hat den kleinsten Stabilitätsbereich. Zwar lässt sich der Stabilitätsbereich durch Verringerung und der Schrittweite T vergrößern, man kann aber durch die Wahl eines „besseren“ Integrationsverfahren trotz dessen höheren Rechenaufwands für einen Rechenschritt eine überproportionale Verbesserung der Genauigkeit erzielen, wodurch der Gesamtrechenaufwand sinkt.

Das Euler-Verfahren spielt praktisch nur noch in der Echtzeitsimulation eine Rolle, da dort noch andere Randbedingungen zu beachten sind.

Probleme bereitet das Euler-Verfahren vor allem bei schwach gedämpften schwingungsfähigen Systemen (z. B. in der Mechanik – siehe Pole), da sich die „**Lücken**“ zwischen Stabilitätsbereich und imaginärer Achse bei Reduzierung der Schrittweite nur langsam schließen.



5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

2. Rückwärts-Euler-Verfahren:

Das Rückwärts-Euler-Verfahren hat einen sehr großen Stabilitätsbereich der auch große Bereiche der rechten s-Halbebene einschließt. Dies bedeutet aber, dass auch sehr viele naturgemäß (kontinuierlich) instabile Systeme als stabil simuliert werden, was bei einer Untersuchung der Systemeigenschaften natürlich u. U. genauso unerwünscht ist wie der umgekehrte Fall.

Im Vergleich zum Vorwärts-Euler-Verfahren ist das Rückwärts-Euler-Verfahren somit nicht genauer, bzw. es werden bezogen auf die s-Ebenen genauso viele instabile Systeme fälschlicher Weise als stabil simuliert, wie beim Vorwärts-Euler-Verfahren stabile Systeme fälschlicher Weise instabil simuliert werden.

Nichtsdestotrotz ist das Rückwärts-Euler-Verfahren wegen seines sehr großen Stabilitätsbereich gerade in der *Echtzeitsimulation* sehr beliebt.

5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

3. Trapezregel :

Der Stabilitätsbereich der Trapezregel umfasst genau die linke s-Halbebene, was bedeutet, dass genau solche Systeme stabil simuliert werden, die es als kontinuierliche Systeme auch sind und instabile ebenso instabil simuliert werden. Die Trapezregel-Verfahren ist somit bzgl. des Stabilitätsbereichs das ideale Verfahren.

Hinweis 1: Integrationsverfahren höher Ordnung (s. u.) haben zwar einen kleineren Stabilitätsbereich gegenüber der Trapezregel, sind aber dennoch genauer.

Viele Integrationsverfahren sind so angelegt, dass ihr Stabilitätsbereich möglichst weite Teile der linken s-Halbebene einschließt und sich an die imaginäre Achse anschmiegt ohne in die rechte s-Halbebene zu ragen.

Hinweis 2: Der Stabilitätsbereiche der impliziten Verfahren (Rückwärts-Euler, Trapezregel) wird durch die für die Auflösung der impliziten Form notwendigen Maßnahmen (z. B. zusätzliche numerische Schritte) je nach Umsetzung beeinträchtigt, d.h. i.d.R. reduziert.

5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Prädiktor-Korrektor-Verfahren:

Für das x_{k+1} auf der „rechten“ Seite der impliziten Iterationsvorschrift wird näherungsweise ein Wert durch ein anderes (explizites) Verfahren berechnet:

Prädiktorschritt: $x_{k+1}^P = x_k + T f(x_k, u_k)$ (expliziter Euler)

Korrektorschritt: $x_{k+1} = x_k + \frac{T}{2} \left(f(x_{k+1}^P, u_{k+1}) + f(x_k, u_k) \right)$ (Trapezregel)

Die Bezeichnung als „Prädiktor-Korrektor-Verfahren“ rührt offensichtlich von der zweistufigen Berechnung mit einem *prädizierenden* ersten Schritt und einem *korrigierenden* zweiten Schritt her.

Hinweis: Das oben dargestellte Verfahren wird meist als **Euler-Heun-Verfahren** teilweise aber auch als **modifiziertes Euler-Verfahren** bezeichnet.

5. Digitale Simulation – 5.2 Klassische Integrationsverfahren

Verfahrensfehler oder kurz „Fehler“, auch numerischer Fehler o. globaler Fehler: Akkumulierte Abweichung zwischen der exakten Lösung. und dem Verfahren zum Zeitpunkt τ unter der Annahme, dass mit exakten Werten gestartet wurde:

$$e = e_g(\tau) = e_{g,k}^{(T)}(\tau) = |x(t_k) - x_k| = |x(\tau) - x_k|$$

$$\text{mit } x(0) = x_0 \quad \text{und} \quad \tau = k \cdot T$$

Ordnung / Fehlerordnung:

Die Ordnung eines Verfahrens p besagt, dass der globale Fehler e bei veränderlichem T maximal mit T^p anwächst, also gilt:

$$e \leq c \cdot T^p \quad \text{mit einer Konstante } c.$$

Hinweise:

- Die Definition beinhaltet keine absolute Aussage über e , da c im allgemeinen unbekannt ist und außer vom Verfahren auch vom Anfangswertproblem abhängt.

5. Digitale Simulation – 5.3 Einschrittverfahren

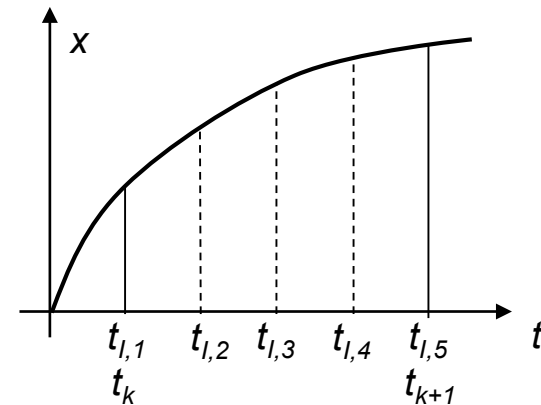
Einschrittverfahren:

Numerische Integrationsverfahren für Anfangswertprobleme bei denen die näherungsweise Integration über einen Abtastschritte unter Verwendung von p Stützstellen erfolgen werden als p -stufige **Einschrittverfahren** bezeichnet:

$$x(t_{k+1}) = x(t_k) + \int_{t_k}^{t_{k+1}} f(\tau) d\tau \approx x(t_k) + \Psi(x(t_k), u(t_{l,1}), u(t_{l,2}), \dots, u(t_{l,p}), t_{l,1}, t_{l,2}, \dots, t_{l,p})$$

Hinweis:

Die Stützstellen können gleichverteilt sein, müssen es aber nicht und es können Stützstellen mit den Intervallgrenzen t_k und t_{k+1} zusammenfallen, müssen es aber ebenso nicht.



Achtung: Die Interpretation von p -stufige als p Stützstellen ist in sofern missverständlich, als dass hier Stützstellen an identischen Stelle liegen können (s.u.). Es handelt sich also bei p -stufig vielmehr um eine p -fache Auswertung der Funktion f .

5. Digitale Simulation – 5.3 Einschrittverfahren

Quadraturformel:

- Ein Vorschrift zur numerischen Berechnung eines Integrals wird in der numerischen Mathematik als **Quadraturformel** bezeichnet.
- Die Approximation des Integrals bei numerischen Verfahren für Anfangswertproblemen (Ein- oder Mehrschrittverfahren) basiert auf Quadraturformeln.
- Die allgemeine Form ist einer Quadraturformeln ist

$$\int_a^b f(\tau) d\tau \approx (b - a) \sum_k c_k f(t_k)$$

wobei $f(t_k)$ Stützstellen mit $a < t_k < b$ und c_k geeignet gewählte Koeffizienten sind.

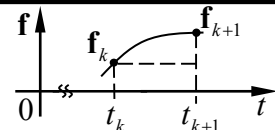
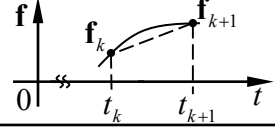
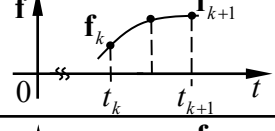
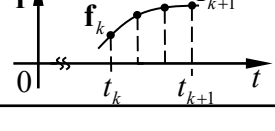
- Generell basieren Quadraturformeln auf geeigneten, analytisch integrierbaren Approximationen der Funktion f , wobei häufig interpolierende Polynomen verwendet werden (interpolierenden Quadraturformel).
- Es gibt sehr viele Quadraturformeln, die sich bzgl. ihrer Approximationsgenauigkeit und dem Rechenaufwand unterscheiden.
- Basierend auf den unterschiedlichen Quadraturformeln lassen sich unterschiedliche Verfahren für Anfangswert Probleme ableiten.

5. Digitale Simulation – 5.3 Einschrittverfahren

Bekannte Quadraturformeln:

Newton-Cotes-Formeln:

Eine Newton-Cotes-Formel vom Grad p benutzt ein interpolierendes Polynom mit $p + 1$ äquidistant verteilten Stützstellen. Dies führt für niedrigere Grade auf die bereits bekannten Verfahren:

Grad	Bezeichnung	Koeffizienten	
0	Rechteckregel	$c_k = 1$	
1	Trapezregel	$c_k = \frac{1}{2}; \frac{1}{2}$	
2	Simpson-Regel	$c_k = \frac{1}{6}; \frac{4}{6}; \frac{1}{6}$	
3	Pulcherrima-Regel	$c_k = \frac{1}{8}; \frac{3}{8}; \frac{3}{8}; \frac{1}{8}$	

5. Digitale Simulation – 5.3 Einschrittverfahren

Integrationsverfahren für Anfangswertprobleme:

Integrationsverfahren für Anfangswertprobleme basieren zwar auf den Quadraturformeln, sind durch diese aber noch nicht eindeutig festgelegt, da die Stützstellen bei Anfangswertproblemen nicht Apriori bekannt sind.

→ Eine Quadraturformel definiert ein *implizites* Verfahren für Anfangswertprobleme.

Lösung: Prädiktor-Korrektorverfahren: Da bei der Ordnung $p \geq 2$ mehrere Stützstellen unbekannt sind, müssen mindestens $p - 1$ Prädiktor-Schritte hintereinander ausgeführt werden.

5. Digitale Simulation – 5.3 Einschrittverfahren

Einige Beispiele:

- **Euler-Heun:** Basierend auf der Trapezregel als Quadraturformel und mit Euler-Verfahren als Prädiktor:

$$\begin{aligned}x^P_{k+1} &= x_k + T f(x_k, u_k) \\x_{k+1} &= x_k + \frac{T}{2} \left(f(x^P_{k+1}, u_{k+1}) + f(x_k, u_k) \right)\end{aligned}$$

(2-stufig, Ordnung 1)

- **Mittelpunktregel** : (auch modifizierter expliziter Euler oder Halbschrittverfahren)

$$x^P_{k+\frac{1}{2}} = x_k + \frac{T}{2} f(x_k, u_k) \quad (\text{expliziter Euler auf Intervallmitte})$$

$$x_{k+1} = x_k + T f\left(x^P_{k+\frac{1}{2}}, u_k\right) \quad (\text{Rechteckregel bei Intervallmitte})$$

(2-stufig, Ordnung 2)

5. Digitale Simulation – 5.3 Einschrittverfahren

- **Heun-Verfahren:** Abwandlung des Euler-Heun mit anderer Stützstelle

$$x^P = x_k + \frac{2}{3}Tf(x_k, u_k) \quad \left(\text{expliziter Euler auf } \frac{2}{3}\text{-Intervall}\right)$$

$$x_{k+1} = x_k + \frac{T}{4} \left(3f\left(x^P, u_{k+\frac{2}{3}}\right) + f(x_k, u_k) \right)$$

(2-stufig, Ordnung 2)

- **Runge-Kutta 3.Ordnung:** Basierend auf Simpson-Regel:

$$x^{P1} = x_k + \frac{1}{2}Tf(x_k, u_k) \quad \left(\text{expliziter Euler auf Intervallmitte}\right)$$

$$x^{P2} = x_k - Tf(x_k, u_k) + 2Tf\left(x^{P1}, u_{k+\frac{1}{2}}\right) \quad \left(\text{Intervallende}\right)$$

$$x_{k+1} = x_k + \frac{T}{6} \left(f(x_k, u_k) + 4f\left(x^{P1}, u_{k+\frac{1}{2}}\right) + f(x^{P2}, u_{k+1}) \right)$$

(3-stufig, Ordnung 3)

5. Digitale Simulation – 5.3 Einschrittverfahren

Die oben benutzte Schreibweise des Runge-Kutta-3- Verfahrens ist unübersichtlich und berücksichtigt nicht die Mehrfachauswertung von f mit identischen Argumenten. Eine günstigere und besser algorithmisierbare Darstellung ist:

Runge-Kutta 3.Ordnung:

$$q_1 = f(x_k, u_k)$$

$$q_2 = f\left(x_k + \frac{T}{2}q_1, u_{k+\frac{1}{2}}\right)$$

$$q_3 = f(x_k - Tq_1 + 2Tq_2, u_{k+1})$$

$$x_{k+1} = x_k + \frac{T}{6}(q_1 + 4q_2 + q_3)$$

5. Digitale Simulation – 5.3 Einschrittverfahren

Die allgemeine Darstellung eines expliziten Einschrittverfahrens lautet damit:

$$\mathbf{q}_1 = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$$

$$\mathbf{q}_2 = \mathbf{f}(\mathbf{x}_k + a_{21}T\mathbf{q}_1, \mathbf{u}(t_k + Tb_2))$$

$$\mathbf{q}_3 = \mathbf{f}(\mathbf{x}_k + a_{31}T\mathbf{q}_1 + a_{32}T\mathbf{q}_2, \mathbf{u}(t_k + Tb_3))$$

⋮

$$\mathbf{q}_p = \mathbf{f}(\mathbf{x}_k + a_{p1}T\mathbf{q}_1 + a_{p2}T\mathbf{q}_2 + \cdots + a_{p,p-1}T\mathbf{q}_{p-1}, \mathbf{u}(t_k + Tb_p))$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + T(c_1\mathbf{q}_1 + c_2\mathbf{q}_2 + c_3\mathbf{q}_3 + \cdots + c_m\mathbf{q}_p)$$

5. Digitale Simulation – 5.3 Einschrittverfahren

Runge-Kutta-Tableau:

Der Übersichtlichkeit halber werden die Verfahren in sogenannten Runge-Kutta-oder Butcher-Tableaus dargestellt.

p -stufiges Einschrittverfahren:

b_2	a_{21}				
b_3	a_{31}	a_{32}			
$\vdots$	$\vdots$	$\vdots$	$\ddots$		
b_p	a_{p1}	a_{p2}	$\dots$	$a_{p,p-1}$	
	c_1	c_2	$\dots$	c_{p-1}	c_p

- Die Koeffizienten b_i definieren die Stützstellen.
- Die Koeffizienten c_i definieren die Quadraturformel.
- Die Koeffizienten a_i definieren die Prädiktormechanismen.

5. Digitale Simulation – 5.3 Einschrittverfahren

Euler-Heun:

1	1
	1/2 1/2

Euler:

2/3	2/3
	1/4 3/4

Mittelpunktsregel:

1/2	1/2
	0 1

Simpsonverfahren:

1/2	1/2
1	1/2 1/2
	1/6 4/6 2/6

Runge-Kutta-3

1/2	1/2
1	-1 2
	1/6 4/6 1/6

Runge-Kutta- 4

1/2	1/2
1/2	0 1/2
1	0 0 1
	1/6 2/6 2/6 1/6

3/8-Regel (Kutta)

1/3	1/3
2/3	-1/3 1
1	1 -1 1
	1/8 3/8 3/8 1/8

5. Digitale Simulation – 5.3 Einschrittverfahren

Schrittweitensteuerung:

- Bei Integrationsverfahren mit variabler Schrittweite wird die Schrittweite T in Abhängigkeit des lokalen Verfahrensfehlers automatisch gesteuert.
- Dadurch lässt sich die Simulation wegen der im Allgemeinen größeren Schrittweite T gegenüber Verfahren mit konstanter Schrittweite T beschleunigen, während gleichzeitig eine hinreichende Berücksichtigung von kritischen Stellen (wie z. B. Schaltvorgänge) durch lokale Reduzierung der Schrittweite erfolgt.

Zur Schrittweitensteuerung über den Verfahrensfehler existieren prinzipiell zwei Ansätze:

1. Parallelrechnung eines Integrationsverfahren mit zwei unterschiedlichen Schrittweiten und der darauf basierender Schätzung des lokalen Fehlers.
2. Parallelrechnung mit unterschiedlicher Ordnung bzw. Stufen p und der darauf basierender Schätzung des lokalen Fehlers.

Im Allgemeinen wird auf das zweite Verfahren zurückgegriffen, das gegenüber der ersten Lösung ein reduzierten Rechenaufwand beinhaltet, da Zwischengrößen wiederverwendet werden können.

5. Digitale Simulation – 5.3 Einschrittverfahren

Dies heißt, dass implizite Anteile in der Differenzengleichung enthalten sind

$$\mathbf{q}_1 = \mathbf{f} \left(\mathbf{x}_k + a_{11}T\mathbf{q}_1 + a_{12}T\mathbf{q}_2 + \cdots + a_{1,p-1}T\mathbf{q}_{p-1} + a_{1,p}T\mathbf{q}_p, \mathbf{u}(t_k + Tb_1) \right)$$

$$\mathbf{q}_2 = \mathbf{f} \left(\mathbf{x}_k + a_{21}T\mathbf{q}_1 + a_{22}T\mathbf{q}_2 + \cdots + a_{2,p-1}T\mathbf{q}_{p-1} + a_{2,p}T\mathbf{q}_p, \mathbf{u}(t_k + Tb_2) \right)$$

$$\mathbf{q}_3 = \mathbf{f} \left(\mathbf{x}_k + a_{31}T\mathbf{q}_1 + a_{32}T\mathbf{q}_2 + \cdots + a_{3,p-1}T\mathbf{q}_{p-1} + a_{3,p}T\mathbf{q}_p, \mathbf{u}(t_k + Tb_3) \right)$$

⋮

$$\mathbf{q}_p = \mathbf{f} \left(\mathbf{x}_k + a_{p1}T\mathbf{q}_1 + a_{p2}T\mathbf{q}_2 + \cdots + a_{p,p-1}T\mathbf{q}_{p-1} + a_{p,p}T\mathbf{q}_p, \mathbf{u}(t_k + Tb_p) \right)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + T(c_1\mathbf{q}_1 + c_2\mathbf{q}_2 + c_3\mathbf{q}_3 + \cdots + c_m\mathbf{q}_p)$$

und bedeutet, dass in jedem Simulationsschritt vor Berechnung der Ausgangsgleichung ein Gleichungssystem in $\mathbf{q} = [\mathbf{q}_1 \dots \mathbf{q}_p]^T$ gelöst werden muss. Das kann prinzipiell mit den in

Kap. 1.4 vorgestellten Verfahren erfolgen, bewirkt aber auch eine Erhöhung der

5. Digitale Simulation – 5.3 Einschrittverfahren

Um das Lösen des Gleichungssystems zu vereinfachen, werden fast ausschließlich die sogenannten *diagonal implizites Runge–Kutta Verfahren* (**DIRK**–Verfahren) eingesetzt. Hierbei hat Runge-Kutta- oder Butcher-Tableau die Diagonalform

b_1	a_{11}				
b_2	a_{21}	a_{22}			
b_3	a_{31}	a_{32}			
$\vdots$	$\vdots$	$\vdots$	$\ddots$		
b_{p-1}	$a_{p-1,1}$	$a_{p-1,2}$	$\dots$	$a_{p-1,p-1}$	
b_p	a_{p1}	a_{p2}	$\dots$	$a_{p,p-1}$	$a_{p,p}$
	c_1	c_2	$\dots$	c_{p-1}	c_p

so dass die Gleichung zeilenweise gelöst werden können da sie jeweils nur ein (neues) implizites Element enthalten. Ein Verfahren wird als **SDIRK**–Verfahren (einfach diagonal implizit) bezeichnet, wenn die Diagonalelemente den gleichen Wert aufweisen.

5. Digitale Simulation – 5.3 Einschrittverfahren

Einfache Beispiele:

Impliziter Euler

1	1
	1

Mittelpunktregel:

1/2	1/2
	1

Trapezregel

0	0	
1	1/2	1/2
	1/2	1/2

Verfahren höher Ordnung:

Radau-Verfahren
(3 Ordnung 2 stufig):

0	0	0
2/3	1/3	1/3
	1/4	3/4

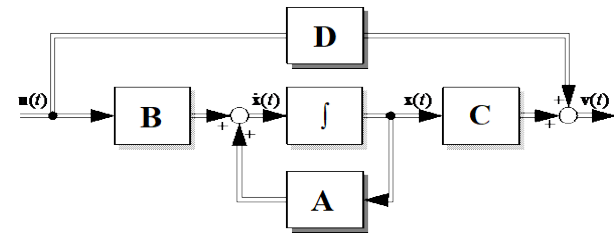
1/3	5/12	-1/12
1	3/4	1/4
	3/4	1/4

5. Digitale Simulation – 5.3 Einschrittverfahren

Lineare Systeme:

Bei linear-zeitinvarianten System (LZI/LTI) liegt eine besonders einfache Form der Systemfunktion f (und der Ausgangsfunktion g) vor

$$\begin{aligned}\dot{x} &= f(x, u) = Ax + Bu \\ y &= g(x, u) = Cx + Du\end{aligned}$$



was in vielen Fällen eine gesonderte Behandlung vor allem der impliziten Verfahren zulässt. Wir wollen dies noch einmal genauer betrachten, um einen Zusammenhang zwischen der numerischen Behandlung von Anfangswertproblemen und der bei linearen Systemen möglichen analytischen Vorgehensweise zu verdeutlichen.

Mit der Transitionsmatrix $\Phi(t)$ gilt:
$$x(t) = \Phi(t - t_0)x_0 + \int_{t_0}^t \Phi(t - \tau)Bu(\tau) d\tau$$

Es sind unterschiedliche Methoden zur Berechnung der Transitionsmatrix bekannt. Bekannter Weise kann die Transitionsmatrix als auf Matrizen erweiterte Exponentialfunktion geschrieben werden:

$$\Phi(t) = \exp(At) = e^{At}$$

5. Digitale Simulation – 5.3 Einschrittverfahren

Diese Darstellung von Φ rechtfertigt sich eigentlich ausschließlich aus der Reihenentwicklung der Exponentialfunktion:

$$\exp(\Gamma) = e^{\Gamma} = I + \Gamma + \frac{1}{2}\Gamma^2 + \dots + \frac{1}{n!}\Gamma^n + \dots = \sum_{k=0}^{\infty} \frac{1}{k!}\Gamma^k$$

Diese Definition kann eigentlich immer angewendet werden, unabhängig ob Γ skalar, vektorwertig, eine Matrix, ein Tensor, komplex ... ist.

Für die $\Phi(t) = e^{At}$ erhält man mit

$$\Phi(t) = e^{At} = I + At + \frac{1}{2}(At)^2 + \dots + \frac{1}{n!}(At)^n + \dots = \sum_{k=0}^{\infty} \frac{1}{k!}(At)^k$$

die Ableitung

$$\dot{\Phi}(t) = A + A \cdot At + A \cdot \frac{1}{2}(At)^2 + \dots + A \cdot \frac{1}{n-1!}(At)^{n-1} + \dots$$

$$= A \cdot \sum_{k=0}^{\infty} \frac{1}{k!}(At)^k = A \cdot e^{At} = A \cdot \Phi(t)$$

5. Digitale Simulation – 5.3 Einschrittverfahren

Mit $\dot{\Phi}(t) = A \cdot \Phi(t)$ erfüllt Φ also die homogene Differentialgleichung $\dot{x} = Ax$ und ist wegen des Eindeutigkeitssatzes die Transitionsmatrix des Anfangswertproblems $\dot{x} = Ax + Bu$ mit $x(t_0) = x_0$.

Schrittweise Lösung:

Will man das Anfangswertproblem schrittweise (im Sinne eines Einschrittverfahrens) für die Zeitpunkte $t_1, t_2, t_3, t_4 \dots$ lösen, so erhält man unter der Annahme, dass die Eingangsgröße u für die jeweiligen Intervalle näherungsweise konstant ist

$$\begin{aligned}
 x_{k+1} &= \Phi(t_{k+1} - t_k)x_k + \int_{t_k}^{t_{k+1}} \Phi(t_{k+1} - \tau)Bu(t_k) d\tau \\
 &= \Phi(T)x_k + \underbrace{\Phi(T) \cdot \int_0^T \Phi(-\tau)d\tau \cdot Bu_k}_{\substack{\text{(Herleitung über Reihenentwicklung)}}} \quad (\text{mit } T = t_{k+1} - t_k) \\
 &= \Phi(T)x_k + \Phi(T) \cdot (I - \Phi(-T))A^{-1} \cdot Bu_k \\
 &= \Phi(T)x_k + (\Phi(T) - I)A^{-1} \cdot Bu_k = \Phi(T)x_k + H(T)u_k
 \end{aligned}$$

5. Digitale Simulation – 5.3 Einschrittverfahren

Näherungsweise Berechnung von Φ :

In der Praxis wird Φ häufig näherungsweise durch Abbruch der Reihenentwicklung für die Exponentialfunktion nach dem n -ten Glied ermittelt:

$$\Phi(t) = e^{At} \approx I + At + \frac{1}{2}(At)^2 + \dots + \frac{1}{n!}(At)^n = \sum_{k=0}^n \frac{1}{k!}(At)^k$$

Bei Abbruch nach dem linearen Glied erhält man mit $\Phi(t) \approx I + At$ für die schrittweise Lösung des Anfangswertproblems

$$\begin{aligned} \mathbf{x}_{k+1} &= (I + AT)\mathbf{x}_k + ((I + AT) - I)A^{-1} \cdot B\mathbf{u}_k \\ &= \mathbf{x}_k + T\mathbf{A}\mathbf{x}_k + T\mathbf{B}\mathbf{u}_k \\ &= \mathbf{x}_k + T(\mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k) = \underline{\mathbf{x}_k + \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)} \end{aligned}$$

Die so gewonnene Gleichung für ein Einschrittverfahren entspricht aber wieder dem *expliziten Euler-Verfahren*.

Auch viele andere Einschrittverfahren ergeben sich für lineare Systeme eins-zu-eins aus abgebrochenen Reihenentwicklungen.

5. Digitale Simulation – 5.3 Einschrittverfahren

Setzt man das lineare System $\dot{x} = Ax + Bu$ in das Euler-Heun-Verfahren ein, erhält man

$$x^P_{k+1} = x_k + TA x_k + T B u_k$$

$$x_{k+1} = x_k + \frac{T}{2} \left(A x^P_{k+1} + B u_{k+1} + A x_k + B u_k \right)$$

oder zusammengefasst

$$\begin{aligned}
 x_{k+1} &= x_k + \frac{T}{2} (A(x_k + T A x_k + T B u_k) + B u_{k+1} + A x_k + B u_k) \\
 &= x_k + T A x_k + \frac{1}{2} T^2 A^2 x_k + T B u_k + \frac{1}{2} T^2 A B x_k \quad (\text{mit } u_{k+1} \equiv u_k) \\
 &= \underbrace{\left(I + T A + \frac{1}{2} T^2 A^2 \right)}_{\text{Reihenentwicklung von } \Phi(t) = e^{At} \text{ bis zum quadratische Glied}} x_k + \underbrace{\left[\left(I + T A + \frac{1}{2} T^2 A^2 \right) - I \right]}_{\text{Reihenentwicklung von } \Phi(t) = e^{At} \text{ bis zum quadratische Glied}} A^{-1} B u_k
 \end{aligned}$$

Reihenentwicklung von $\Phi(t) = e^{At}$ bis zum quadratische Glied

Das Euler-Heun-Verfahren lässt sich also mit einer kleinen Einschränkung bzgl. der Eingangsgröße auf eine Reihenentwicklung für Φ bis zur Ordnung 2 zurückführen.

5. Digitale Simulation – 5.4 Mehrschrittverfahren

Mehrschrittverfahren:

Im Unterschied zu Einschrittverfahren werden bei Mehrschrittverfahren auch ältere Zustandswerte als x_k für die Berechnung des neuen Zustandswerts x_{k+1} herangezogen:

$$x_{k+1} = x_k + k(x_{k+1}, x_k, x_{k-1}, \dots, x_{k-n}, \dots)$$

Der allgemeine Ansatz ist

$$\sum_{n=-1}^l a_n x_{k-n} = T \sum_{n=-1}^l b_n f(x_{k-n})$$

Mehrschrittverfahren beruhen wie Einschrittverfahren auf passenden Quadraturformeln.

Hinweis 1: Ist der Koeffizient $b_{-1} = 0$, dann lässt sich die Formel direkt nach x_{k+1} auflösen und man erhält ein explizites Verfahren, ansonsten ein implizites.

Hinweis 2: Eine theoretisch aus denkbare Formulierung eines Mehrschrittverfahrens mit Zwischenstufen im (vgl. mehrstufige Einschrittverf.) ist unüblich.

5. Digitale Simulation – 5.4 Mehrschrittverfahren

Adams-Bashforth-Verfahren:

Explizite Mehrschrittverfahren die auf der Lagrangeschen Integrationsformel beruhen werden Adams-Bashforth-Verfahren genannt. Für konstante Schrittweite T lauten sie:

Adams-Bashforth-2:
$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{T}{2} (3\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) - \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}))$$

Adams-Bashforth-3:
$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{T}{12} (23\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) - 16\mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + 5\mathbf{f}(\mathbf{x}_{k-2}, \mathbf{u}_{k-2}))$$

Adams-Bashforth-4:
$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{T}{24} (55\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) - 59\mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + 37\mathbf{f}(\mathbf{x}_{k-2}, \mathbf{u}_{k-2}) - 9\mathbf{f}(\mathbf{x}_{k-3}, \mathbf{u}_{k-3}))$$

Hinweis1: Als „Adams-Bashforth-1“ erhält man das explizite Euler-Verfahren, was natürlich kein Mehrschrittverfahren darstellt aber auch in der Reihe der Adams-Bashforth-Verfahren liegt.

Hinweis2: Adams-Bashforth-Verfahren nutzen ein Integrationspolynom des Stützstellen außerhalb des zu integrierenden Bereichs liegen. Dies führt zu einem relativ großen lokalen Fehler.

5. Digitale Simulation – 5.4 Mehrschrittverfahren

Adams-Moulton-Verfahren:

In gleicher Weise lassen sich implizite Mehrschrittverfahren durch die Lagrangesche Integrationsformel ableiten. Diese werden Adams-Moulton-Verfahren genannt. Für konstante Schrittweite T lauten sie:

Adams-Moulton-2:
$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{T}{12} (5\mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}) + 8\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) - \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}))$$

Adams-Moulton-3:
$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{T}{24} (9\mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}) + 19\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) - 5\mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{f}(\mathbf{x}_{k-2}, \mathbf{u}_{k-2}))$$

Hinweis: Als „Adams-Moulton-0“ erhält man das implizite Euler-Verfahren und als „Adams-Moulton-1“ die Trapezregel (s.o.).

5. Digitale Simulation – 5.4 Mehrschrittverfahren

Adams-Bashforth-Moulton-Verfahren:

Zur Lösung der impliziten Adams-Moulton-Verfahren werden diese in der Praxis mit Adams-Bashforth-Verfahren nach der Prädiktor-Korrektor-Methode kombiniert. Das resultierende Verfahren wird dann als *Adams-Bashforth-Moulton-Verfahren* bezeichnet.

Adams-Bashforth-Moulton-3:

$$x^P_{k+1} = x_k + \frac{T}{12} (23f(x_k, u_k) - 16f(x_{k-1}, u_{k-1}) + 5f(x_{k-2}, u_{k-2}))$$

$$x_{k+1} = x_k + \frac{T}{24} (9f(x^P_{k+1}, u_{k+1}) + 19f(x_k, u_k) - 5f(x_{k-1}, u_{k-1}) + f(x_{k-2}, u_{k-2}))$$

Hinweis1: In der Literatur findet man für die Adams-Bashforth-Moulton-Verfahren auch die Bezeichnung PECE (Predictor-Evaluation-Corrector-Evaluation).

Hinweis2: Zur Verbesserung der Genauigkeit werden teilweise mehrere Prädiktor-Korrektor-Schritte hintereinander ausgeführt. (PECECE).

5. Digitale Simulation – 5.4 Mehrschrittverfahren

Bewertung der Mehrschrittverfahren:

Vorteil:

- Durch die Wiederverwendung gespeicherter Werte der Systemfunktion f an alten Stützstellen ($f(x_{k-1}, u_{k-1}), f(x_{k-2}, u_{k-2}), \dots$) ist der Rechenaufwand für ein Mehrschrittverfahren im Vergleich zu einem Einschrittverfahren mit „vergleichbaren“ numerischen Eigenschaften geringer:

Adams-Bashforth-Moulton-3: 2 Auswertung von f pro Rechenschritt

Runge-Kutta-3 bzw. 4 : 3 bzw. 4 Auswertung von f pro Rechenschritt.

(Für einen genauen Vergleich müsste man jedoch den lokalen Fehler genauer betrachten.)

Je höher die Ordnung desto besser wird das Verhältnis gegenüber den Einschrittverfahren.

5. Digitale Simulation – 5.4 Mehrschrittverfahren

Nachteile:

- Schon in Kapitel 1.4 wurde darauf hingewiesen, dass Mehrschrittverfahren eine Anlaufrechnung z. B. mittels eines Einschrittverfahren benötigen, bis genügend alte Stützstellen zum Starten des Mehrschrittverfahrens vorliegen. Dies ist nicht nur beim Start der Simulation sondern auch bei Strukturwechseln im Modell notwendig.
- Die Schrittweitensteuerung ist bei Mehrschrittverfahren erheblich aufwendiger. Grundsätzlich basiert sie wie bei den Einschrittverfahren z. B. auf einer Abschätzung des lokalen Fehlers durch Kombination zweier Verfahren unterschiedlicher Güte. Problematisch ist, dass die Koeffizienten a_k und b_k bei variabler Schrittweite T nicht konstant sind und in jedem Schritt über ein passendes Interpolationspolynom neu bestimmt werden müssen.

Dem Vorteil durch die geringer Zahl von Auswertung von f bzgl. steht also ein Mehraufwand bei der Anlaufrechnung und der Schrittweitensteuerung entgegen!

5. Digitale Simulation – 5.4 Mehrschrittverfahren

Fazit:

Ob ein Mehrschrittverfahren gegenüber einem vergleichbaren Einschrittverfahren Vorteile bzgl. der Simulationszeit bietet hängt vor allem auch von der zugrundeliegenden Simulationsaufgabe ab:

- Mehrschrittverfahren bieten sich an wenn die Systemfunktion f sehr aufwendig in der Auswertung ist (z. B. viel nicht-rationale Terme enthält).
- Bei Systemen mit häufigen Strukturwechseln (z. B. schaltende Leistungselektronik) mindert sich der Vorteil der Mehrschrittverfahren.
- Mehrschrittverfahren sind auch nicht grundsätzlich gut geeignet für steife System (Problem: Schrittweitensteuerung). Ausnahme Gear-Verfahren (s. u.).

5. Digitale Simulation – 5.5 Übersicht der Verfahren

Übersicht der Verfahren I: (teilweise aus [1.1]) ¹ Mehrschrittverfahren ² nur für explizite Verfahren

Verfahren	Ordn.	expl./impl.	Stab.	MS ¹	Fkt.-Auswertung ²	Steife Syst.
Expl. Euler	1	e			1	
Impl. Euler	1	i	L			ja
Semiimpl. Euler	1	(e)			1(+J)	
Expl./impl. Euler	1	(e)			1	
Trapezregel	2	i	A			bedingt
Mod. Euler	2	e			2	
Euler Heun	2	e			2	
Heun	2	e			2	
Expl. Runge-Kutta-3	3	e			3	
Expl. Runge-Kutta-4	4	e			4	
Expl. Runge-Kutta-5	5	e			6	

5. Digitale Simulation – 5.5 Übersicht der Verfahren

Übersicht der Verfahren II: (teilweise aus [1.1])

Verfahren	Ordn.	expl./impl.	Stab.	MS ¹	Fkt.-Auswertung ²	Steife Syst.
Impl. Runge-Kutta-3	3	i	L			ja
Impl. Runge-Kutta-4	4	i	L			ja
Rosenbrock-3	3	(e)	L			ja
Adams-Bashforth-2	2	e		ja	1	
Adams-Bashforth-3	3	e		ja	1	
Adams-Bashforth-4	4	e		ja	1	
Adams-Moulton-3	3	i		ja		
Adams-Moulton-4	4	i		ja		
Adams-B.-M.-3	3	e		ja	2	
Adams-B.-M.-4	4	e		ja	2	

¹ Mehrschrittverfahren ² nur für explizite Verfahren

5. Digitale Simulation – 5.5 Übersicht der Verfahren

Übersicht der Verfahren III: (teilweise aus [1.1])

Verfahren	Ordn.	expl./ impl.	Stab.	MS ¹	Fkt.-Aus- wertung ²	Steife Syst.
Gear-2	2	i	A	ja		ja
Gear-3	3	i		ja		
Gear-4	4	i		ja		

¹ Mehrschrittverfahren ² nur für explizite Verfahren

5. Digitale Simulation – 5.6 Spezielle Probleme

Nichtlineare Systeme:

„Starke“ Nichtlinearitäten beeinflussen in erheblichem Maß die Simulation dadurch, dass

- „relative“ harmlos aussehende Probleme für den Integrationsalgorithmus als extrem steif erscheinen,
- theoretische Genauigkeit nicht erreichbar sind weil zugrundeliegende Funktionen teilweise nicht differenzierbar sind (und keiner Lipschitz-Bedingung genügen),
- diskrete Ereignisse auftreten, die bei falscher Behandlung zu sehr „unphysikalischen“ Simulationsergebnissen führen.

5. Digitale Simulation – 5.6 Spezielle Probleme

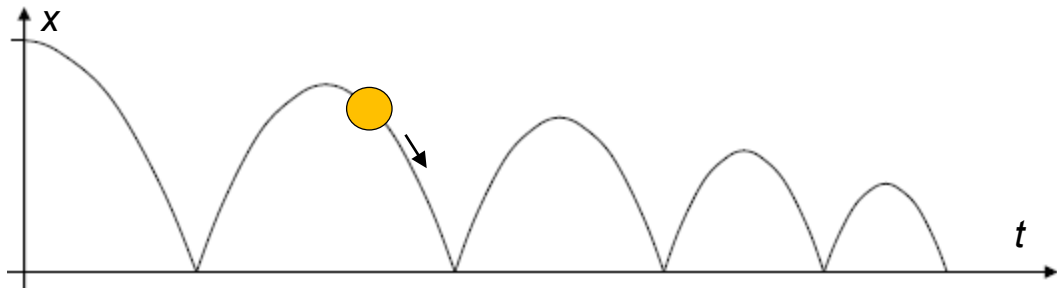
Diskrete Ereignisse (Zustandsereignisse):

Diskrete Ereignisse treten in einer Simulation auf, wenn aufgrund des Erreichens von *Schwell-, End- oder Sättigungswerten* oder einer *bestimmten Relation zweier Zustandsvariablen* eine schaltende (nicht zeitlich differenzierbare) Änderung einer Größe oder ihrer zeitlichen Ableitung auftritt.

Solche diskrete Ereignisse stellen sich oft als elementare Struktur- oder Topologiewechsel im zu simulierenden System dar. Werden diese diskrete Ereignisse zeitlich nicht (relativ) exakt berücksichtigt, kann dies zu physikalisch unsinnigen Simulationsergebnissen.

Beispiel 1: Bouncing Ball

Mögliches Problem:
Ball springt unter
die Erdoberfläche



Beispiel 2: Schaltungssimulation

Mögliches Problem: Diode leitet rückwärts

5. Digitale Simulation – 5.6 Spezielle Probleme

Detektion von Zustandsereignissen (Zero-Crossing-Detection)

Voraussetzung: Variable Schrittweite

1. Erkennung eines Zustandsereignisses im während des letzten Abtastschritts: Z. B. Null-Durchgang der Zustandsgröße x_n (n -tes Element von x).
2. Suche des exakten Ereigniszeitpunkts T_0 . Problemstellung:

$$x_{n,k+1} = \mathbf{e}_n \cdot \mathbf{x}_{k+1} = \mathbf{e}_n \cdot \Psi(\mathbf{x}_k, T_0) = 0 \quad \text{mit} \quad \mathbf{e}_n = [0 \quad \dots \quad 1 \quad \dots \quad 0]$$

(Problemstellung event. Umformulieren)

→ Numerische Lösungsverfahren Verfahren:

- Bisection-Methode
- Regula Falsi
- Newton-Iteration

Abbruchbedingung der Iteration:
Gewünschte Genauigkeit.

3. Fortsetzen des Integrationsverfahrens mit veränderten Größen bzw. veränderter Struktur.

Beispiel: Regula Falsi

